AgentShield Architecture Handbook

A compact technical report for the Monday demonstration - tech stack, shared security flow, and speaker-wise domain architecture.

CORE IDEA	AgentShield is a security gateway between an agent's requested action and the controlled tool that can execute it. Policy decides first; approval and exact-action verification happen before execution.

1. Tech stack at a glance

The complete stack is intentionally small: a browser dashboard, one Node.js API, an embedded deterministic policy engine, PostgreSQL persistence, and controlled adapters for each domain.

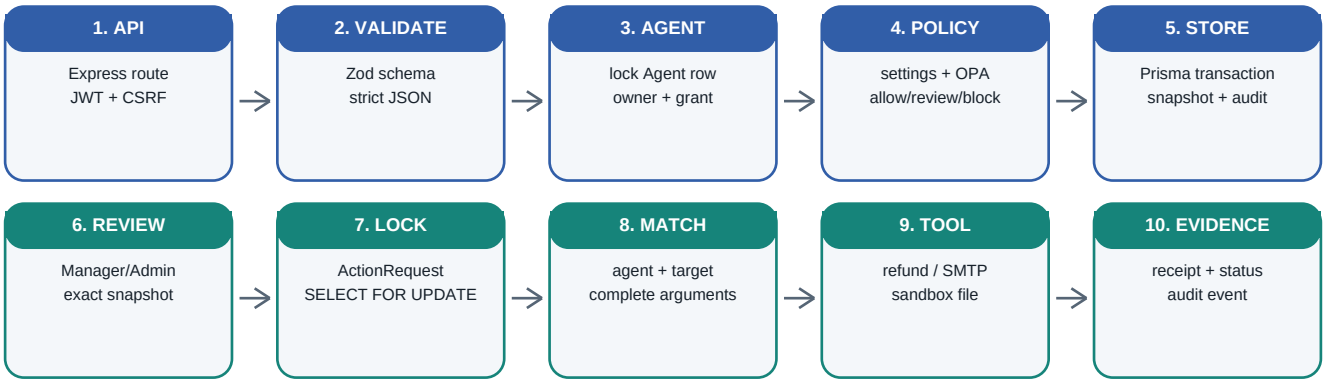
USER INTERFACE	React 19 | Vite 7 | Material UI 7
API AND SECURITY	Node.js | Express 5 | Zod 4 | JWT + CSRF + roles
POLICY ENGINE	OPA Rego compiled to WebAssembly - embedded inside Node.js
DATA AND TOOLS	PostgreSQL + Prisma 6.12 | SMTP/Nodemailer | contained file tool
QUALITY AND RUNTIME	Vitest + Supertest | demo supervisor | 4-second dashboard polling

Technology	Where we use it	Why it matters
React 19 + Material UI 7	Login, domain forms, manager queue, policy editors, audit and receipts	A clear role-aware interface; React renders message/file text safely as text.
Vite 7	Frontend development server and production build	Fast local startup; proxies /api to Express during development.
Node.js + Express 5	REST API and orchestration layer	Central place for authentication, policy, approval and execution checks.
Zod 4	Strict request and settings schemas	Rejects extra fields, invalid paths, bad emails and oversized content early.
OPA Rego + Wasm	Refund, email and sandbox decisions	Deterministic, testable rules embedded in Node - no separate OPA server.
PostgreSQL + Prisma 6.12	Users, sessions, agents, requests, decisions, reviews, execution and audit	Durable snapshots and transaction/locking support.
Nodemailer 10.1.7	Optional authenticated SMTP email	TLS transport with a durable one-attempt reservation.
Contained filesystem tool	Real sandbox READ/WRITE/CHANGE/DELETE	Maps logical workspace/ paths into a dedicated demo folder.
Vitest + Supertest	Policy, API, SMTP and sandbox tests	Checks rules, permissions, tampering, replay and failure cases.

IMPORTANT	There is no LLM in the authorization path. OPA produces deterministic decisions; identity, ownership, approval snapshots, replay protection and execution integrity are enforced by backend code.

2. How the backend works for every domain

Refund, Email and Sandbox are not three separate applications. They enter the same secure backend pipeline; only the OPA input and final tool adapter change.



What each backend layer actually does

Layer	Simple meaning	Important implementation
Routes + middleware	Who may call this endpoint?	actionRoutes.js applies requireAuth + requireCsrf; pending/review/preview also require Manager/Admin.
Controller + Zod	Is the HTTP input well formed?	actionController.js validates body, UUID and query; strict schemas reject hidden extra fields.
Submission service	Is this agent currently allowed to request this tool?	Prisma transaction locks Agent; checks ownerId, active and allowedTools before policy.
OPA policy service	What is the deterministic decision?	Embedded Rego/Wasm returns only ALLOW, REQUIRE_APPROVAL or BLOCK using current versioned settings.
Repository	What evidence must survive refresh/restart?	Atomically stores ActionRequest, PolicyDecision and initial audit events in PostgreSQL.
Execution service	Is this still the exact authorized action?	Locks request, rejects replay, deep-compares complete details, then rechecks Agent before tool dispatch.
Domain adapter	What real/simulated effect occurs?	Refund simulator, Nodemailer SMTP adapter, or contained sandbox filesystem adapter.

Why the database structure matters

1 — ActionRequest
Original owner, agent, action, target, arguments and status

2 — PolicyDecision
Decision, reason, policy version and settings snapshot

3 — ApprovalRecord
Reviewer + exact approved action snapshot

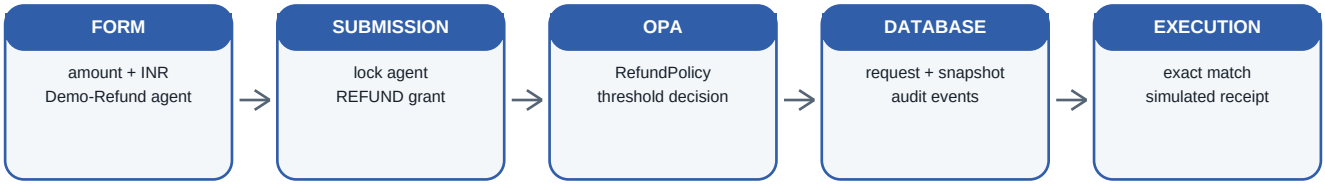
Record	Purpose
ExecutionRecord	One durable receipt/reservation per request ID
AuditEvent	Chronological evidence of every allowed/denied stage

THE MAIN RULE — The tool never trusts the browser's current form. It executes only the saved original authorization or Manager approval snapshot, after current permissions are checked again.

OPA integration: refund.rego, email.rego and sandbox.rego are compiled to checked Wasm assets. opaPolicy.js verifies the manifest, source/Wasm SHA-256 and exact entrypoint at startup. The Wasm runs inside Node.js; no separate policy server or AI model is involved.

3. Refund: from amount to controlled receipt

Risk: an agent may request too much money, change the amount after approval, use an unauthorized refund agent, or replay an already executed request.



Policy and backend implementation together

Amount compared with saved policy	OPA result	Backend behavior
0 < amount <= auto-approve limit	ALLOW	Store AUTHORIZED. Exact saved action can execute.
auto-approve limit < amount <= maximum	REQUIRE_APPROVAL	Store PENDING_APPROVAL. Manager approves the exact amount/target/agent.
amount > maximum	BLOCK	Store BLOCKED. Execution service refuses it.

1 Submission transaction
submissionService locks the Agent row, checks owner/active/REFUND grant, reads RefundPolicy and calls evaluateRefundPolicy.

2 Versioned OPA decision
refund.rego receives only REFUND, INR, amount and current limits. policyService maps its result to the saved status.

3 Atomic evidence
createActionRecord stores request, PolicyDecision, REQUEST_RECEIVED and POLICY_EVALUATED together or rolls everything back.

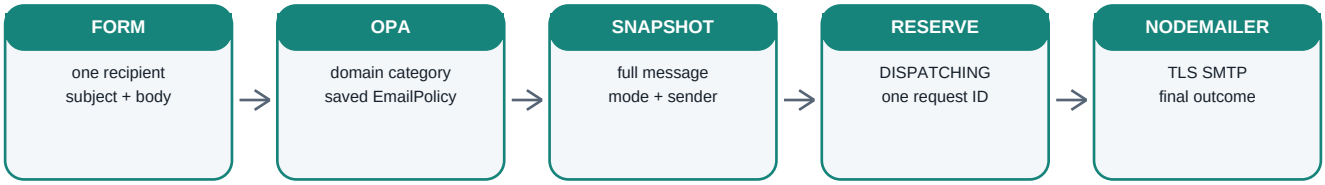
Approval and execution protection

What happens internally: reviewPendingAction accepts only a PENDING_APPROVAL request and stores reviewer identity plus the exact action snapshot. executeAction later chooses that snapshot, compares agentId, action, target and complete arguments, and rejects any changed amount with EXECUTION_MISMATCH. The request row lock and unique ExecutionRecord stop concurrent/double execution.

HONEST DEMO LIMIT	simulateRefund creates a UUID receipt and audit event but calls no bank or payment provider. The security architecture is real; the financial side effect is intentionally simulated.
SAY IT SIMPLY	'Refund request pehle agent permission check se guzarta hai. OPA current limits ke basis par decision deta hai. Approval exact amount se bind hota hai, aur execution se pehle wahi amount aur agent dobara match hote hain. Duplicate request ID execute nahi ho sakta.'

4. Email: policy approval to real SMTP

Risk: an agent may send sensitive text to an unsafe recipient, modify the message after approval, expose SMTP credentials, or resend after an uncertain network result.



Step 1 - request and domain policy

1 Strict email schema

One normalized address, subject 1-200 chars, plain-text body up to 8,000 UTF-8 bytes. No HTML, attachments, CC/BCC or extra fields.

2 Agent permission

submissionService locks Demo-Email and requires owner, active status and SEND_EMAIL before evaluating policy.

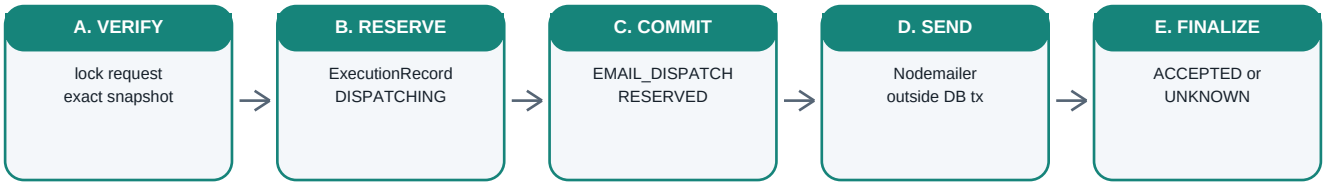
3 Recipient rule

email.rego receives recipientDomain plus EmailPolicy. Internal can ALLOW; blocked always BLOCK; all others need approval or block.

Step 2 - how SMTP was connected safely

Part	Implementation
Configuration	EMAIL_DELIVERY_MODE selects SIMULATED/SMTP. SMTP host, port, user, password and from address stay only in backend/.env.
Transport	Nodemailer uses port 465 immediate TLS or port 587 STARTTLS, TLS 1.2+, certificate checking, timeouts, and disabled file/URL access.
Approved content	executionMode and emailSender are saved by backend. The final message uses only saved recipient, exact subject and exact plain-text body.
Secret exposure	The frontend endpoint returns only mode, readiness, sender and optional demo shortcuts - never host, username or password.

Step 3 - why real sending uses reserve -> send -> finalize



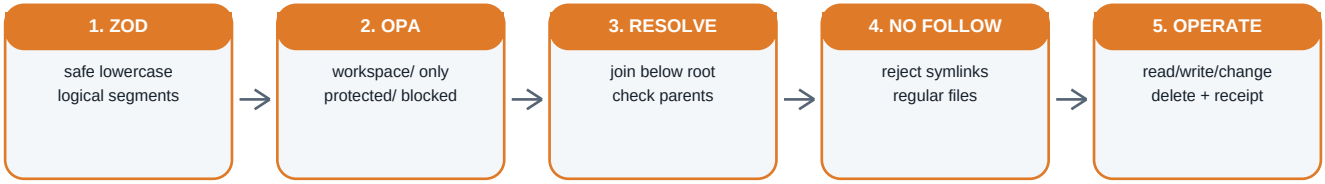
A database rollback cannot unsend an email. Therefore executionService first commits a unique DISPATCHING receipt and stable Message-ID. Only the caller that owns that reservation invokes emailDispatchService. After Nodemailer returns, a second transaction stores ACCEPTED + EXECUTED, or UNKNOWN + FAILED. The reserved request is never automatically resent because a timeout may happen after provider acceptance.

WHAT ACCEPTED MEANS	SMTP ACCEPTED means the provider accepted the recipient. It does not prove inbox delivery, reading, or absence of a later bounce.
SAY IT SIMPLY	'Browser email nahi bhejta; browser sirf request bhejta hai. Backend recipient-domain policy check karta hai, full message approval snapshot mein save karta hai, phir one-attempt reservation ke baad TLS SMTP call hota hai. Isliye changed content aur duplicate send dono control hote hain.'

5. Sandbox: logical workspace to safe folder

Risk: an agent may escape into another laptop folder, use ../ or a symlink, overwrite a file, or change/delete a different file version from the one a manager reviewed.

THE FOLDER IDEA	The API sees logical path workspace/team/notes.txt. sandboxFileTool maps it under backend/demo-workspace/team/notes.txt. The client never sends C:\ paths or receives the physical path.



Defense layers - in simple order

Layer	What the code checks	Why it matters
Schema	1-240 chars, forward-slash lowercase segments; no spaces, hidden names, dot segments or backslashes.	Bad paths are rejected before policy or disk access.
OPA boundary	Must start workspace/; workspace/protected/ always blocked; settings may only narrow access.	A Manager cannot accidentally configure access outside the hard boundary.
Path resolver	Remove workspace/, join below resolved sandboxDirectory, then confirm root + path separator.	The physical target cannot escape backend/demo-workspace.
Filesystem checks	lstat every parent; directories/files cannot be symlinks; open with O_NOFOLLOW where available.	Blocks link-based escape and non-regular targets.

How each real file operation is implemented

Operation	OS behavior	Approval/safety behavior
READ	Open O_RDONLY; read text <= 8,000 bytes.	Allowed only inside current policy boundary.
WRITE	Open O_CREAT | O_EXCL with mode 0600.	Creates a new file; refuses overwrite.
CHANGE	Open existing file O_RDWR, write from offset 0, truncate to new length.	Requires approved SHA-256 and rechecks it before replacement.
DELETE	unlink one existing regular file.	Requires approved SHA-256 and rechecks it before deletion.

Why preview + SHA-256 is important

For CHANGE/DELETE, file-preview returns current text, byte count and SHA-256 - never the physical path. Approval sends that digest; actionRepository rereads the file and refuses FILE_PREVIEW_CHANGED if it differs. The approved digest is stored in the snapshot. sandboxFileTool hashes again immediately before CHANGE/DELETE, so a file modified after approval cannot be silently changed.

REAL EFFECT RULE	Disk changes cannot be rolled back by PostgreSQL. executionService therefore commits a unique SANDBOX_FILE DISPATCHING reservation before touching disk; sandboxDispatchService then records EXECUTED or FAILED. The same request ID is not auto-retried.

SAY IT SIMPLY	'Agent ko laptop path nahi milta; sirf logical workspace/ path milta hai. Backend usko dedicated demo folder ke andar resolve karta hai, symlink aur protected path reject karta hai. CHANGE/DELETE approval current file hash se bind hota hai, isliye stale approval execute nahi hota.'

6. How the complete system fits together

Use this page for the architecture discussion after the live demo. It separates shared security controls from domain-specific behavior and gives the team honest, defensible answers.

Cross-cutting security controls

Control	What it protects	Where enforced
Revocable session + roles	Only logged-in users act; only Manager/Admin reviews	JWT HttpOnly cookie, DB session, current User role
CSRF + origin/client checks	Stops unwanted browser mutations	Auth middleware on POST/PUT/logout
Strict validation	No hidden fields or malformed content	Zod schemas and JSON size limits
Agent registry and tool grants	Agent cannot use an unassigned domain	Submission and execution recheck
Versioned OPA snapshot	Decision remains explainable after settings change	PolicyDecision + audit digest/version
Approval snapshot	Human approval cannot be reused for altered details	ApprovalRecord and exact comparison
Locks + unique execution	Concurrent calls and replay are refused	PostgreSQL row lock + unique requestId
Receipt + audit trail	Team can reconstruct what happened	ExecutionRecord + chronological AuditEvent

Three-person handoff

Order	Presenter	Own this explanation	Handoff line
1	Deepanjan	Shared gateway briefly, then refund thresholds, snapshot and replay protection	'Same secure flow recipient risk par kaise apply hota hai, Sameer dikhayega.'
2	Sameer	Email domain rules, full-message approval and safe SMTP attempt	'Ab Avnish dikhayega ki same architecture filesystem boundary ko kaise protect karti hai.'
3	Avnish	Sandbox containment, operation decisions and preview-hash binding	'Finally, teenon domains common audit and execution gate mein converge karte hain.'

What we should claim - and what we should not

We can confidently claim	We should clearly say is outside this MVP
All three domains use real backend policy, persistence, approval, integrity checks and audit.	AgentShield does not globally intercept Codex, Claude, Gemini or operating-system actions.
Email can simulate or use configured SMTP; sandbox performs real contained file operations.	Refunds are simulated; there is no payment provider or real-money movement.
Policies are deterministic OPA rules, not an AI guess.	Independent agent credentials, approval expiry/revocation, immutable audit storage and public deployment are future work.
The architecture fails closed on invalid policy assets and mismatched execution details.	SMTP provider acceptance is not proof of inbox delivery, and audits are not tamper-proof against a database administrator.

FINAL ARCHITECTURE ANSWER	'AgentShield is a policy-enforcement gateway. It authenticates the user, verifies the registered agent and tool grant, evaluates versioned OPA policy, stores an immutable-for-the-request snapshot, requires human approval when needed, rechecks the exact action immediately before the controlled tool, and records the result. Refund, Email and Sandbox are three adapters on top of that same secure lifecycle.'

Recommended preparation: each presenter reads page 2 plus their own domain page. One person should also be ready to answer from this final page.